

Reviewer Pack — Minimal Cyclic Order and Resolution Proof Width Lower Bound

Entry 9d3b5f3f6b3b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 06:25:17 UTC

Minimal Cyclic Order and Resolution Proof Width Lower Bound

Entry ID: 9d3b5f3f6b3b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 06:25:17 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Cyclic Groups) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every d -regular graph G , the minimal order of a cyclic subgroup generated by a vertex in G is at least the resolution proof width of its associated Tseitin formula φ_G , such that $\min_order(G) \geq w(\varphi_G)$.

Rationale (proposer's reasoning):

The structure of cyclic subgroups in graphs may encode non-trivial information about the complexity of finding satisfying assignments for boolean formulas, suggesting a potential lower bound on resolution proof complexity.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b7f13f7c66c9247f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if a Pearson correlation coefficient of $\min_order(G)$ with $w(\varphi_G)$ is ≥ 0.5 for all graphs, with p -value ≤ 0.05 across 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Cyclic Order AND Resolution Proof Complexity - Cyclic Groups IN BOOLEAN Satisfiability Resolution Proof Width - d-regular graph AND Tseitin formula resolution proof width lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/quant-ph/0405018v2>] Improved Bounds on the Randomized and Quantum Complexity of Initial-Value Problems - [<http://arxiv.org/abs/2503.11238v2>] Computational Complexity of Finding Subgroups of a Given Order - [<http://arxiv.org/abs/1606.01763v2>] A Constructive Proof of Cut Elimination for a System of Full Second Order Logic - [<http://arxiv.org/abs/1103.5740>] Generating and Searching Families of FFT Algorithms - [<http://arxiv.org/abs/1911.04516v1>] Boolean lattices in finite alternating and symmetric groups - [<http://arxiv.org/abs/2406.18700v4>] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if n * d % 2 != 0 or d > n - 1:
        raise ValueError("Invalid parameters for generating a d-regular graph")

    graph = [[] for _ in range(n)]
    degree_count = [0] * n

    def add_edge(u, v):
        if u == v or len(graph[u]) >= d or len(graph[v]) >= d:
            return False
        graph[u].append(v)
        graph[v].append(u)
        degree_count[u] += 1
        degree_count[v] += 1
        return True

    for _ in range(d * n // 2):
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        while not add_edge(u, v):
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
```

```

    return graph

def lcm(a, b):
    return abs(a * b) // math.gcd(a, b)

def min_order(graph):
    orders = []
    for node in range(len(graph)):
        visited = [False] * len(graph)
        queue = [node]
        visited[node] = True
        current_order = 1

        while queue:
            u = queue.pop(0)
            for v in graph[u]:
                if not visited[v]:
                    visited[v] = True
                    queue.append(v)
                    current_order += 1

        orders.append(current_order)

    return lcm(*orders)

def resolution_width(phi_G):
    # Placeholder function to simulate the computation of resolution width
    # For simplicity, we assume a linear relationship with n
    n = len(phi_G)
    return n

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        d = random.randint(1, n - 1)
        graph = generate_d_regular_graph(n, d)
        phi_G = [random.choice([0, 1]) for _ in range(n)]

        min_order_val = min_order(graph)
        width_val = resolution_width(phi_G)

        results.append({
            "n": n,
            "min_order": min_order_val,
            "width": width_val
        })

    correlation_coefficient = 0.0
    for i in range(len(results)):
        for j in range(i + 1, len(results)):
            r_i = results[i]
            r_j = results[j]
            correlation_coefficient += (r_i["min_order"] - r_i["width"]) * (r_j["min_order"] - r_j["width"])

```

```

n_pairs = len(results) * (len(results) - 1) // 2
correlation_coefficient /= n_pairs

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": correlation_coefficient >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.5\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the analysis required to determine the Pearson correlation coefficient and p-value for the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14076
2	preregistration	ollama_remote	glm4:latest	0	0	9128
3	novelty	ollama_remote	glm4:latest	0	0	8233
4	novelty	ollama_remote	glm4:latest	0	0	10000
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49315
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15489
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16444
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12477
9	judge	ollama_remote	glm4:latest	0	0	15267

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150428 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/9d3b5f3f6b3b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/9d3b5f3f6b3b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/9d3b5f3f6b3b.tar.gz* (if generated)